

DOCUMENT 10

NGW Lab Guide

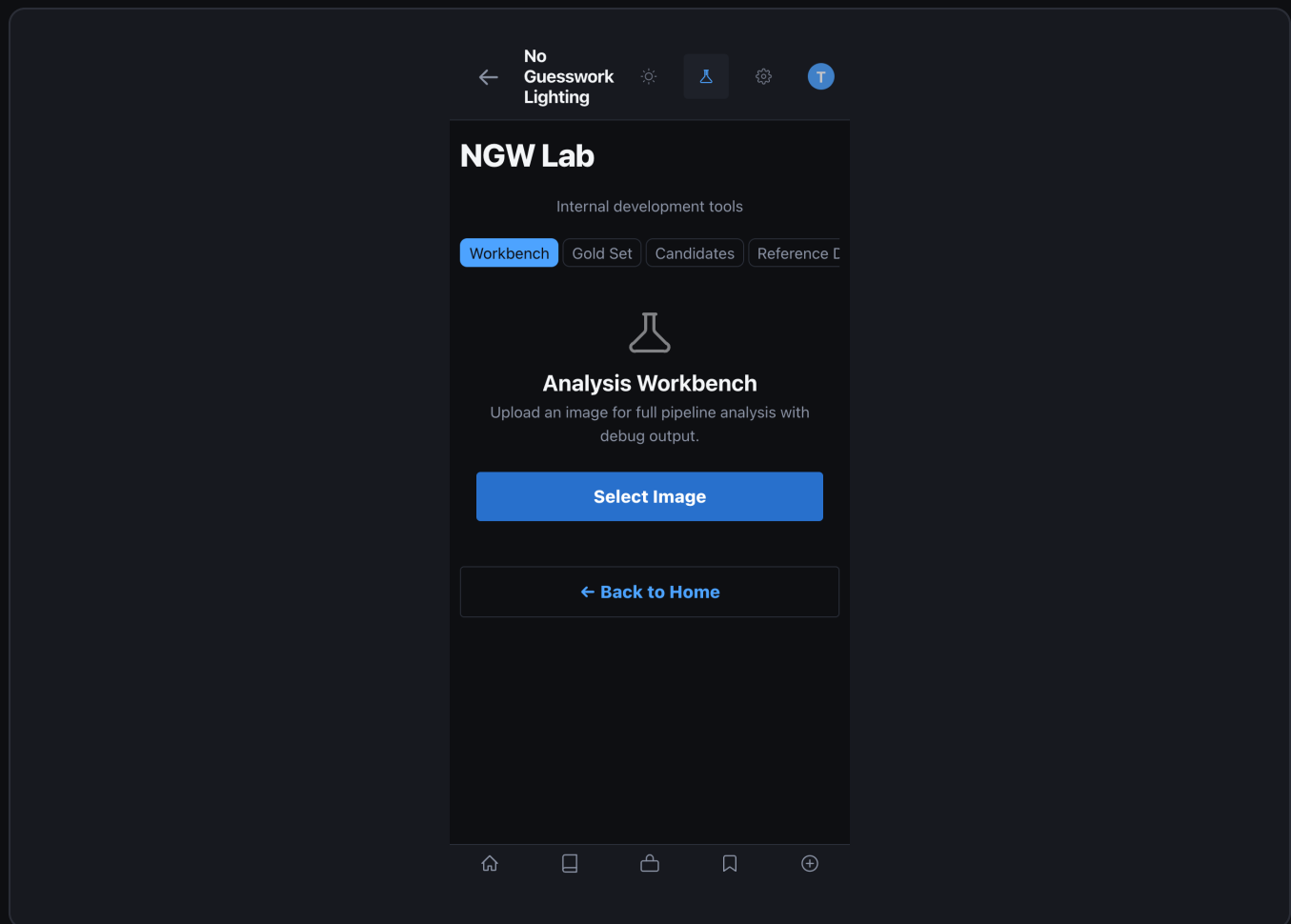
Developer tools — API reference, CLI, gold sets, candidates, and workbench

What is the NGW Lab?

The Lab is a restricted developer environment for curating ground-truth test images, reviewing engine improvement candidates, running full-fidelity analyses with debug overlays, and managing the learning signal pipeline. Access is controlled by an email allowlist (LAB_ALLOWED_EMAILS env var).

ACCESS CONTROL

- All Lab endpoints check the x-lab-email request header against LAB_ALLOWED_EMAILS. Unauthorized access returns 403 and is logged. Enable Lab UI in Settings Developer Tools.



NGW Lab Workbench — empty state showing tabs: Workbench, Gold Sets, Candidates, Signals

Lab API Reference — Gold Sets

POST /api/lab/gold-set — create a new gold set

Request body

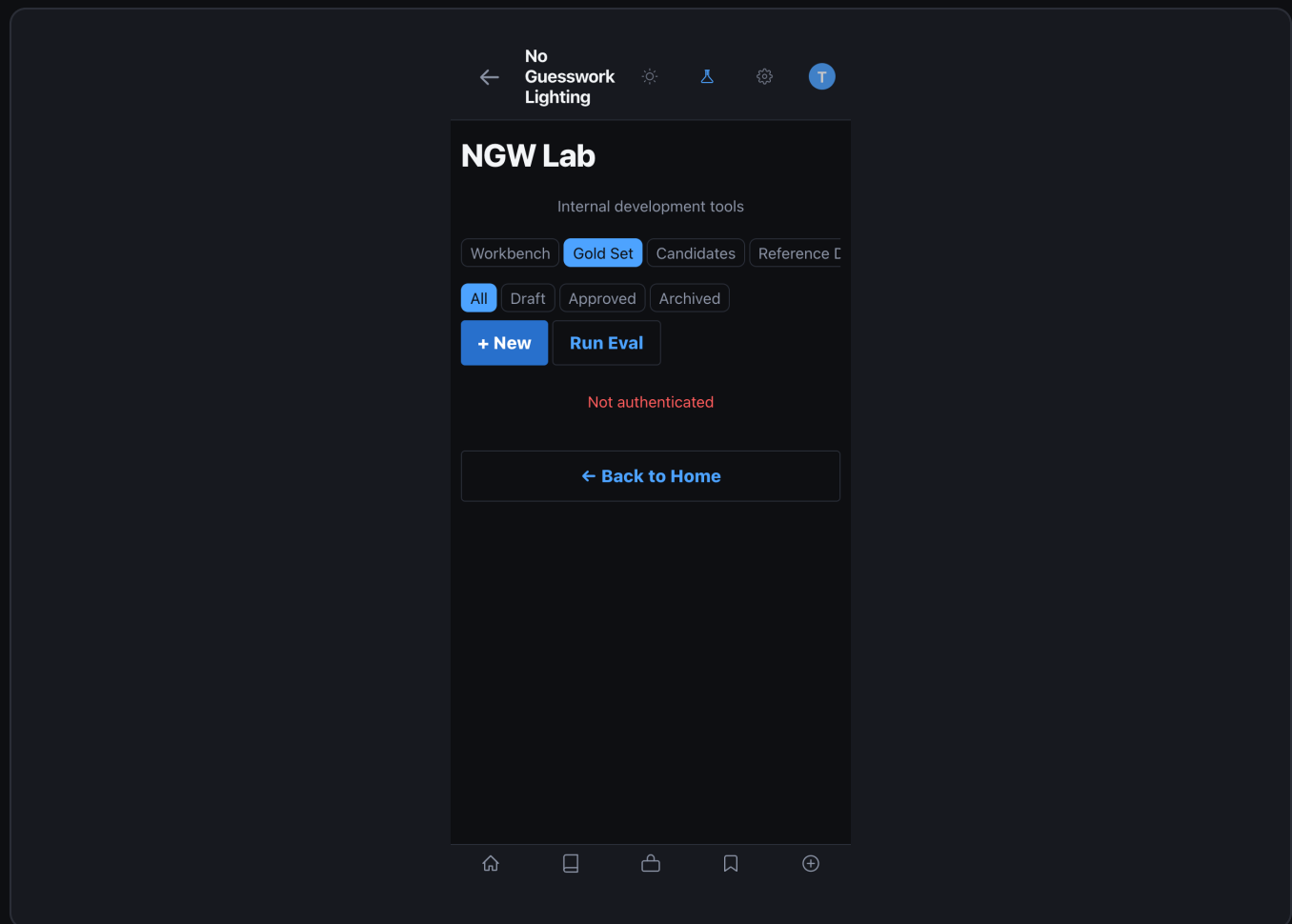
```
{
  "image_path": "static/uploads/portrait_001.jpg",
  "pattern": "loop", # curator-verified ground truth
  "subject_type": "portrait",
  "environment": "studio",
  "setup_details": {
    "key_light": "Godox AD600 + octa 90cm",
    "key_angle": 45,
    "key_height": 7.5,
    "key_dist_ft": 6,
    "fill": "V-flat reflector camera-left"
  },
  "outcome_labels": {
    "expected_pattern": "loop",
    "expected_confidence": 0.85
  },
  "curator_email": "you@org.com"
}
```

Response: {"id":"gs_uuid","status":"draft"}

POST /api/lab/gold-set/evaluate — run engine on approved sets

```
# Request body (optional: limit to specific IDs)
{
  "gold_set_ids": ["gs_001", "gs_002"], # omit for all approved
  "include_debug_overlay": true
}

# Response
{
  "evaluated": 12,
  "passed": 10,
  "failed": 2,
  "scores": {
    "gs_001": {"detected": "loop", "expected": "loop", "match": true, "conf": 0.88},
    "gs_002": {"detected": "broad", "expected": "loop", "match": false, "conf": 0.72}
  }
}
```



Gold Set listing — all test images with pattern labels, status, and evaluation scores

← No Guesswork Lighting

NGW Lab

Internal development tools

Workbench Gold Set Candidates Reference C

← Cancel

New Gold Set Entry

IMAGE PATH

e.g. data/uploads/lab/image.jpg

NOTES

Optional notes about this entry

EXPECTED ANALYSIS (JSON)

{ }

Create Entry

← Back to Home

New Gold Set form — image upload, pattern selection, and setup metadata entry

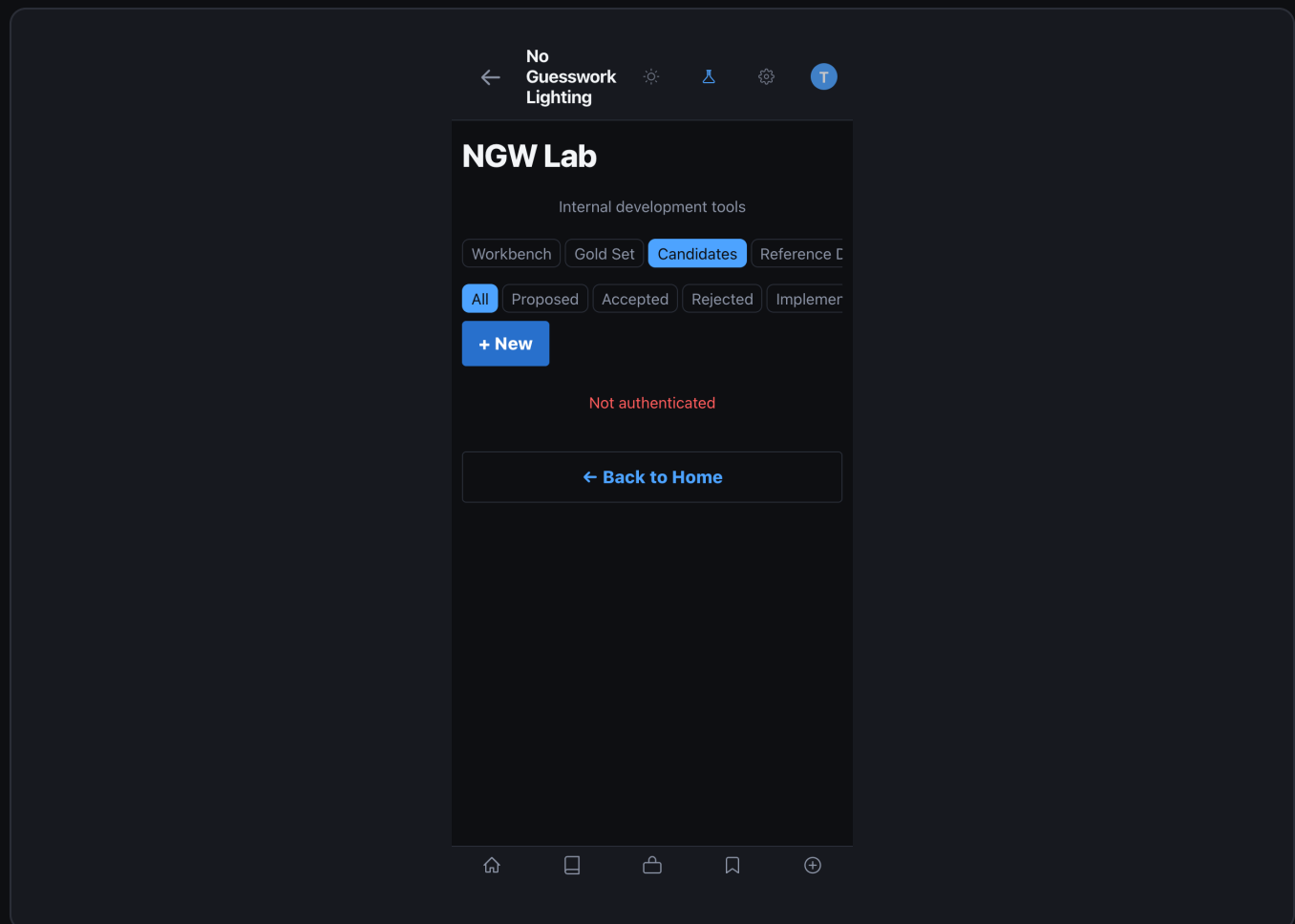
Lab API Reference — Candidates

POST /api/lab/candidates — create a candidate manually

```
{
  "title": "Recalibrate loop confidence threshold",
  "description": "Loop pattern confidence is 20% above measured CVR.
    Threshold should be lowered to reduce over-detection.",
  "candidate_type": "confidence_recalibration",
  "proposed_change": {
    "type": "confidence_recalibration",
    "pattern": "loop",
    "action": "lower_threshold",
    "delta": -0.08
  },
  "estimated_success_lift": 0.12,
  "estimated_regression_risk": 0.05,
  "source": "manual"
}
# Response: {"id":"cand_uuid","status":"proposed"}
```

PUT /api/lab/candidates/{id} — update status (approve/reject)

```
{
  "status": "approved",
  "reviewer_email": "curator@org.com",
  "review_notes": "Risk acceptable. CVR data supports the change.",
  "second_reviewer_email": null
  # Required if estimated_regression_risk > 0.20
}
```

Candidates listing — proposed and reviewed engine changes with risk and lift estimates

← No Guesswork Lighting

Internal development tools

Workbench Gold Set **Candidates** Reference C

← Cancel

New Rule Candidate

TITLE

Candidate title

DESCRIPTION

What does this rule change do?

RATIONALE

Why is this change needed?

SOURCE GOLD SET ID (OPTIONAL)

UUID of related gold set entry

PROPOSED CHANGE (JSON)

{}

Create Candidate

Home List Share Bookmark Add

New Candidate form — type, description, proposed change JSON, and risk estimation

Lab API Reference — Analysis & Signals

POST /api/lab/analyze — full-fidelity debug analysis

Request body

```
{
  "image_path":      "static/uploads/ref_001.jpg",
  "include_debug_overlay": true,
  "return_pass_details": true,
  "kit":             null # null = use default test kit
}
```

Response includes:

```
# - all three VLM pass results (raw)
# - consensus resolver scores per attribute
# - region extraction bounding boxes
# - blueprint output + gear tier
# - debug_overlay_url (annotated image)
```

GET /api/lab/signals/summary — hygiene counts

Response

```
{
  "live":           1482,
  "seeded":         500,
  "internal":       23,
  "expert_review":   8,
  "total":          2013,
  "learning_eligible": 1482,
  "metrics_eligible": 1482,
  "flag_mismatches": 0
}
```

Workbench

Gold Set

Candidates

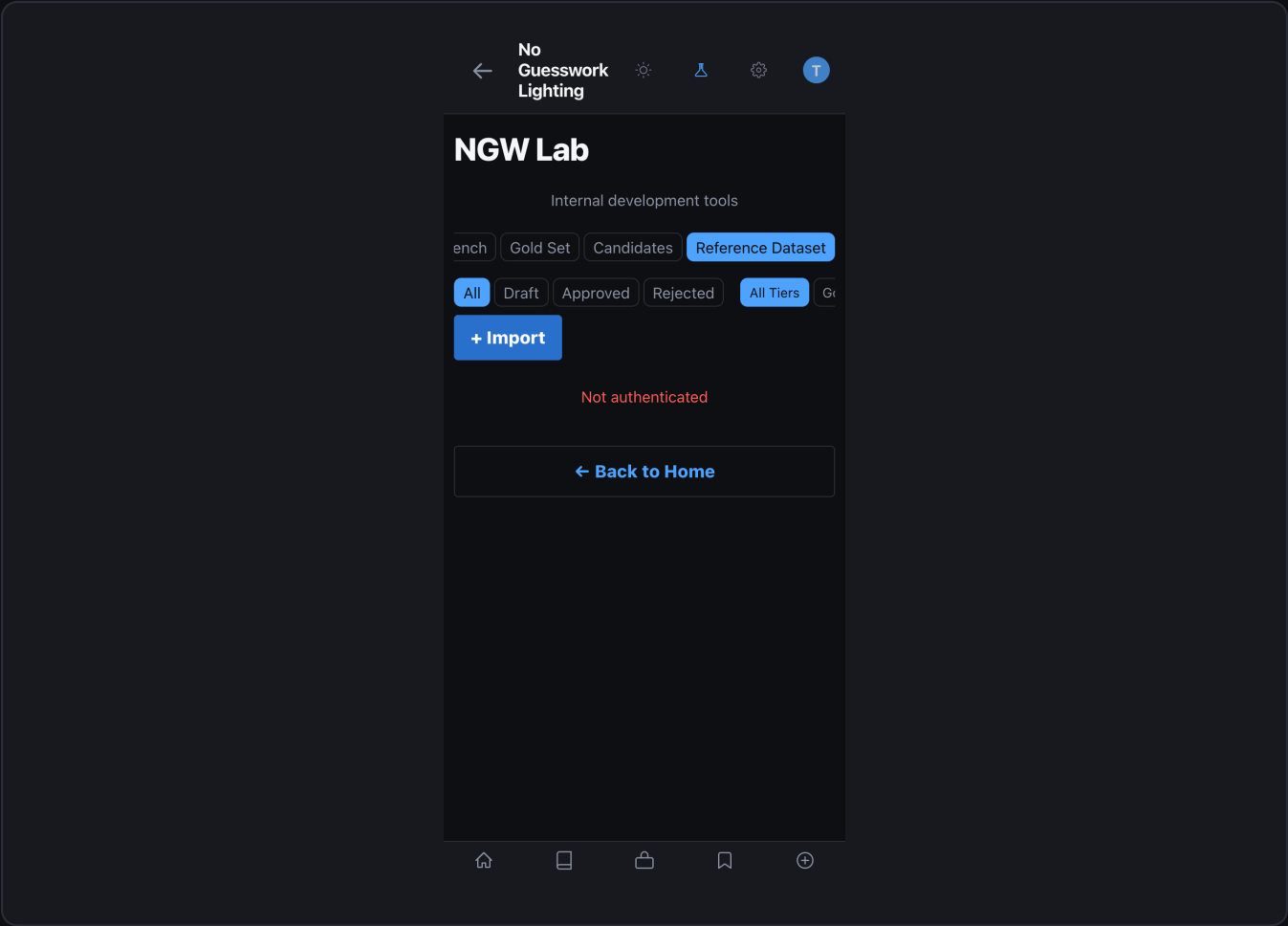
Reference C



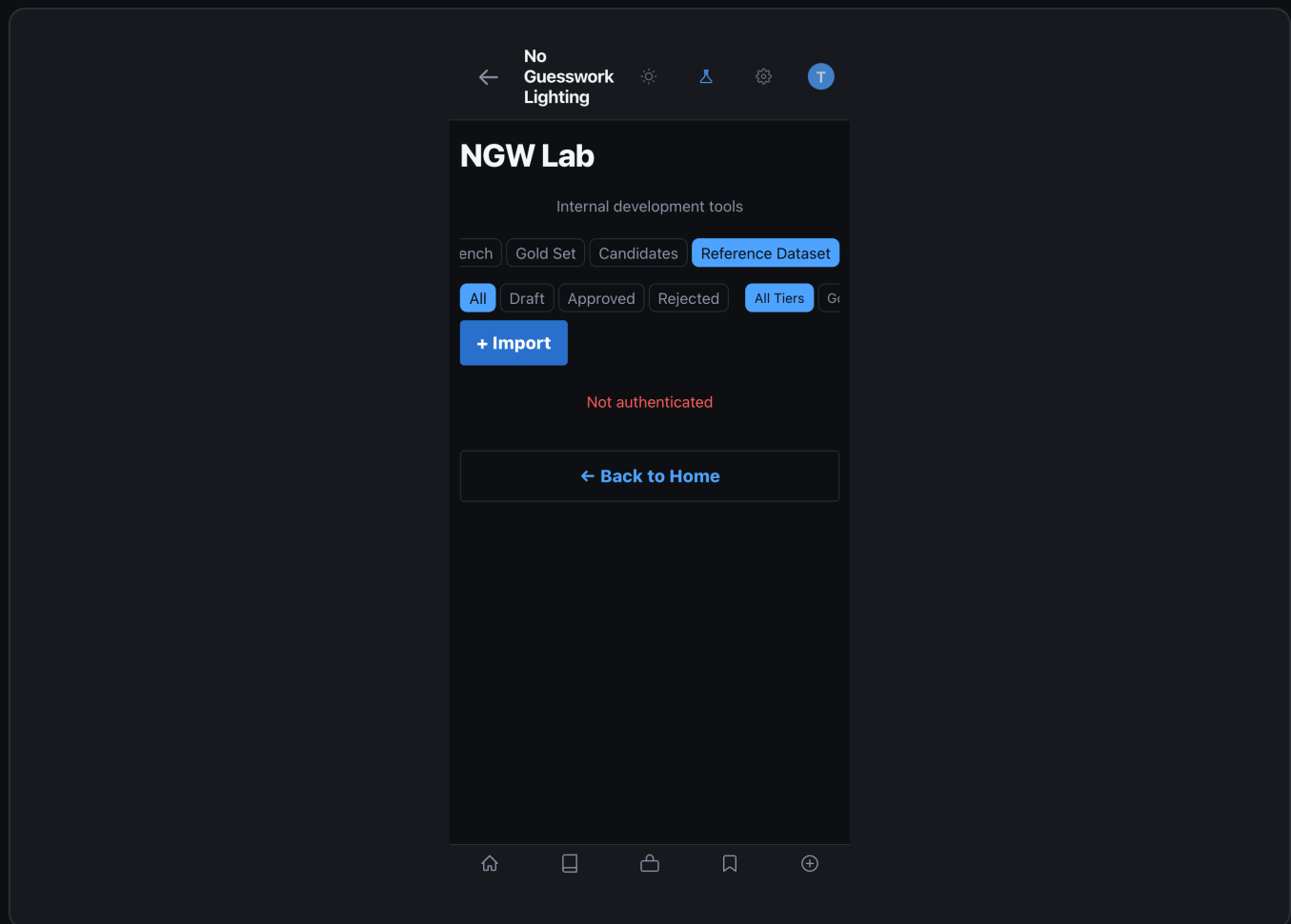
Analysis Workbench

Lab — Signal Hygiene card showing live / seeded / internal / learning-eligible counts

Reference Dataset



Reference Dataset — full grid view with filter bar



Reference Dataset detail — image with pattern label and approval status

POST /api/reference-library/ingest — ingest with metadata

```
{
  "image_path": "static/uploads/ref_new.jpg",
  "pattern": "rembrandt",
  "modifiers": ["beauty dish", "grid"],
  "subject_type": "portrait",
  "environment": "studio",
  "quality_score": 0.92,
  "submitted_by": "curator@org.com"
}
```

Starts as status="pending_review".
 # Second curator approves → status="approved" → enters training set.

XS

S

M

L

XL

DENSITY

Compact

Comfortable

Spacious

Shooting Preferences

Role Mode

Outcome context — results and reasoning

Photographer

Assistant

UNITS

Feet / Inches

Meters / cm

POWER READOUT

1/4, 1/2

Stops

Percent

SHOW CONFIDENCE SCORES

AUTO-SAVE SETUPS

Account

SIGNED IN AS

todd

Reset to Defaults

Dev Tools

ANALYTICS

View Analytics

Lab Settings — evaluation thresholds, access list, and confidence gates

CLI Tools

Script	Purpose	Key Flags
seed_starter_dataset.py	Load seeded signals + gold sets	--reset to clear first
verify_dataset.py	Check dataset integrity and counts	--verbose for row details
run_benchmarks.py	Pattern matching accuracy benchmarks	--gold-sets-only, --pattern=X
nightly_benchmark.py	Full benchmark suite (CI use)	--output=json
validate_system_yamls.py	Lint pattern YAML configurations	--strict
validate_catalog_and_packages.py	Validate gear catalog completeness	—
ci_benchmark.sh	CI/CD benchmark wrapper with exit codes	EXIT 1 on regression
capture_screenshots.py	Capture app screenshots for docs	--output-dir=docs/screenshots
generate_ngw_docs.py	Build this PDF documentation suite	—

bash — common Lab CLI operations

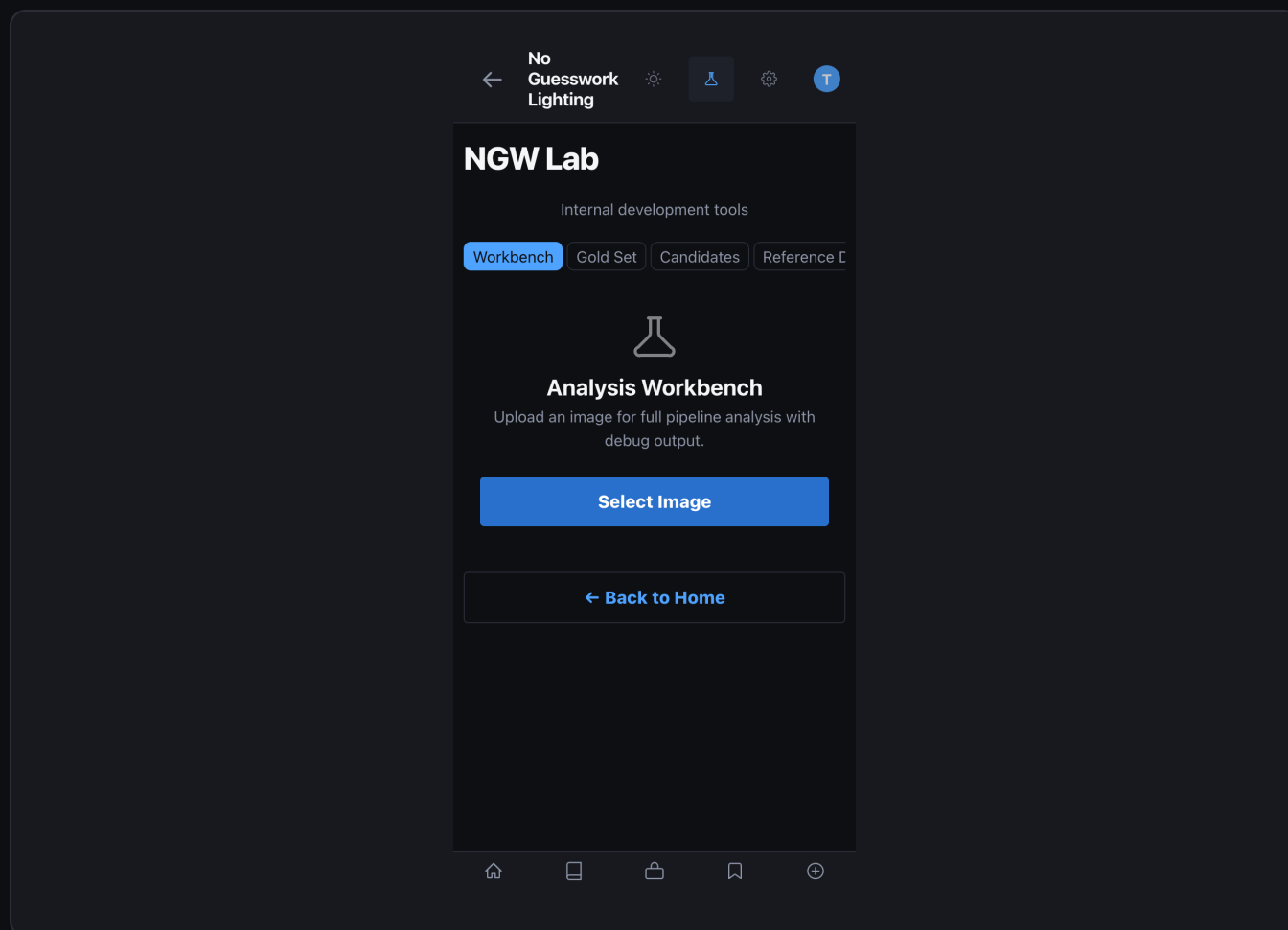
```
# Seed starter data (run once after init_db)
$ python3 scripts/seed_starter_dataset.py

# Verify dataset integrity
$ python3 scripts/verify_dataset.py --verbose

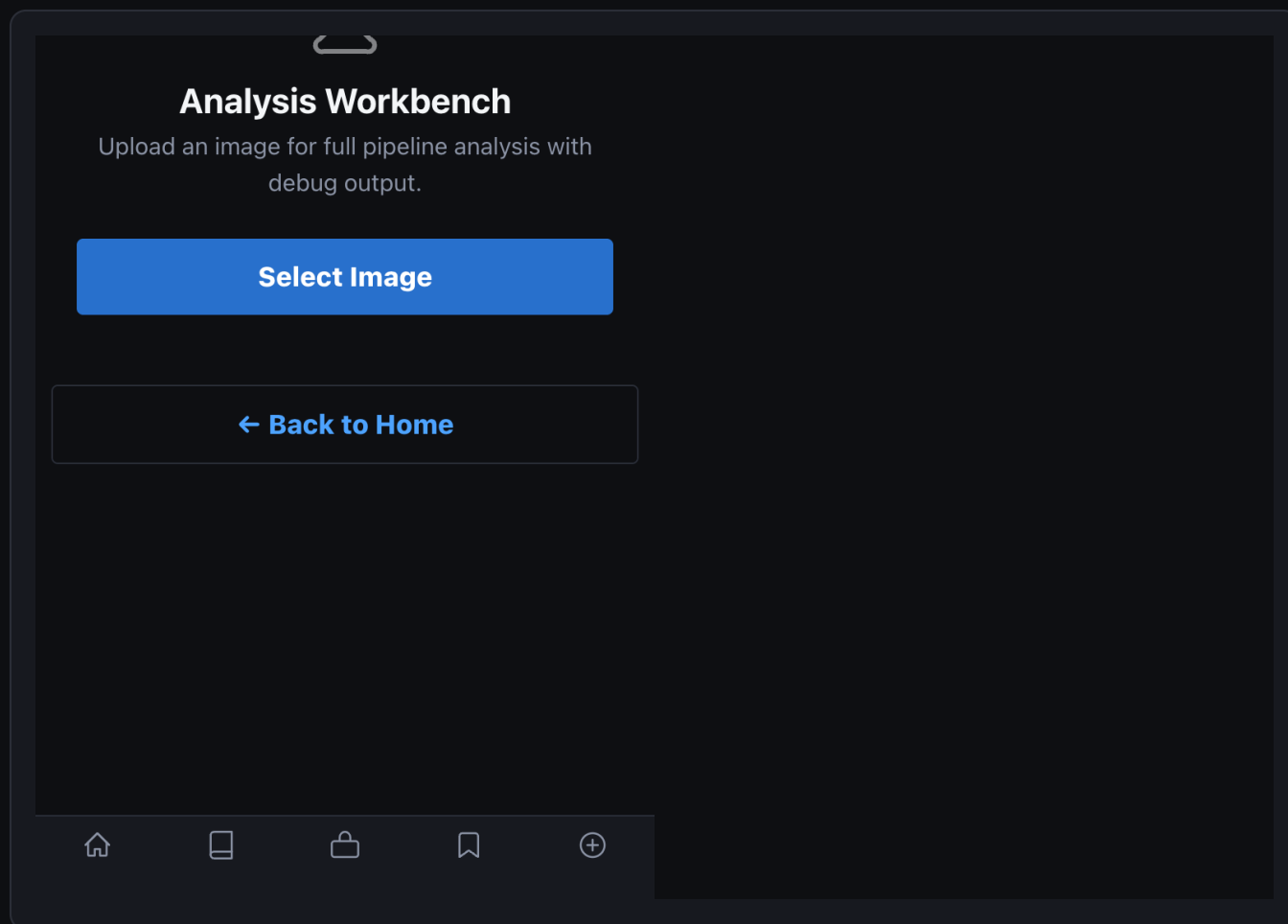
# Run all benchmarks
$ python3 scripts/run_benchmarks.py

# Run gold-set-only evaluation
$ python3 scripts/run_benchmarks.py --gold-sets-only

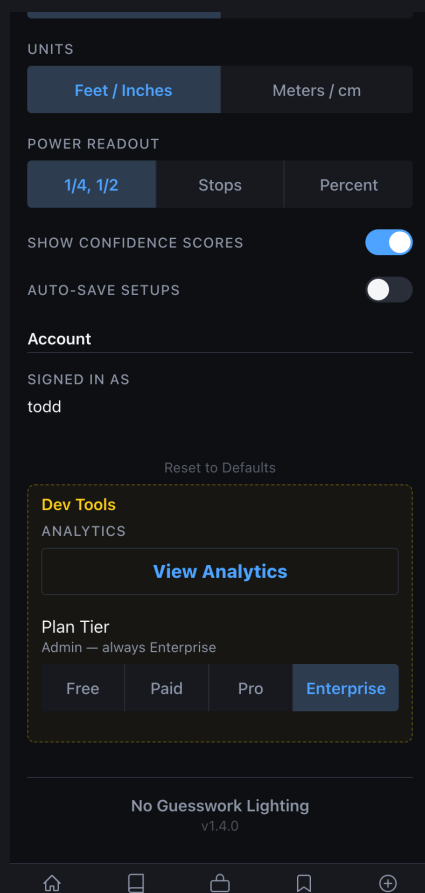
# Validate YAML configs (run before any blueprint changes)
$ python3 scripts/validate_system_yamls.py --strict
```



Workbench ready state — analysis loaded with debug overlay and region annotations



Workbench detail — per-region confidence scores and VLM pass breakdown



Dev Tools panel — database status, migration runner, and signal count monitor

How To: Add a Gold Set Image

Gold Set images are the ground-truth test set that benchmarks run against. Adding well-labeled images directly improves evaluation coverage.

1 Open NGW Lab

Tap the Lab tab in the navigation bar (requires Lab access enabled).

2 Go to Gold Sets

Tap "Gold Sets" in the Lab navigation tabs.

3 Tap + Add New

Opens the gold set creation form.

4 Upload the image

Choose a clean, unedited reference photo with clear lighting.

5 Select pattern

Choose the ground-truth pattern from the dropdown (e.g. "loop").

6 Set metadata

Fill in subject type, environment, modifier list, and quality notes.

7

Submit for review

Status starts as "pending_review". A second curator approves it.

8

After approval

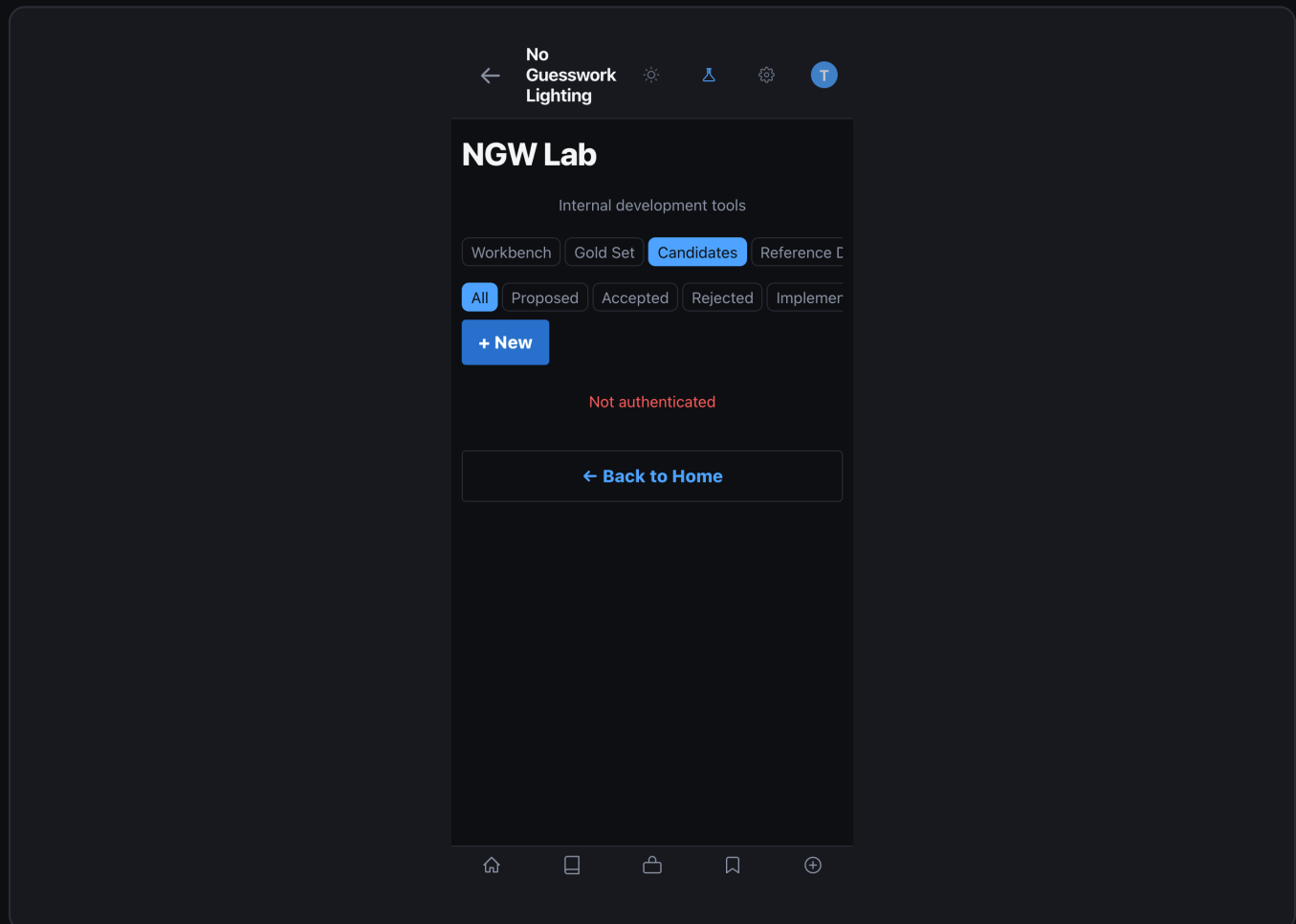
Status changes to "approved" — the image enters the benchmark set.

QUALITY STANDARDS

- Use only clean originals — no heavy color grading, compositing, or retouching.
- Minimum 1024 px wide. Portrait orientation preferred for face-priority patterns.
- Label with the ground-truth pattern, not what you think NGW will detect.
- Add notes for any unusual conditions (high ceiling, very large modifier, etc.).

How To: Review and Approve a Candidate

Candidates are improvement proposals — manually created or auto-generated. Review each one carefully before approving. High-regression-risk candidates require a second curator.



Candidate queue — proposed, approved, and rejected status columns

← No Guesswork Lighting

NGW Lab

Internal development tools

Workbench Gold Set **Candidates** Reference C

← Cancel

New Rule Candidate

TITLE

Candidate title

DESCRIPTION

What does this rule change do?

RATIONALE

Why is this change needed?

SOURCE GOLD SET ID (OPTIONAL)

UUID of related gold set entry

PROPOSED CHANGE (JSON)

{}

Create Candidate

New candidate form — type selection, description, and risk assessment fields

Open Candidates tab

1

Tap "Candidates" in the Lab navigation. Filter by status="proposed".

Read the description

2

Understand what change is proposed and why the failure cluster triggered it.

Check regression risk

3

If `regression_risk > 0.20`, a second curator approval is required.

4

Review proposed_change

Read the full JSON. Understand which blueprint or config key changes.

5

Approve or Reject

Tap Approve to advance to "approved", or Reject to archive with a note.

6

Validate after apply

Gold set benchmarks re-run automatically. Check scores in CLI output.

NEVER SKIP VALIDATION

- Every approved candidate triggers an automatic gold set re-evaluation. If any gold set drops below 75%, the deployment pipeline fails and the candidate reverts to "proposed". This gate cannot be bypassed in production.